Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Building a stack using lists

Today
Welcome to your next lesson: Building a stack using lists.

I'm putting together your lesson based on what you've learned so far, what's coming next, and the best way for you to practice.

Step by step, you’re moving forward.

Ron, quick warmup 🍸

Remember: a stack is “Last In, First Out” — like a stack of plates where the last plate you put on is the first one you take off.

In Python, we’ll build a stack using a plain list.

Key idea 1 (we’ll stick with just this for now):

To push onto a stack, we’ll use list.append(...).

Example (don’t code yet, just read):

Python

```
stack = []          # start empty
stack.append("A")   # push A
stack.append("B")   # push B

# Now stack is ["A", "B"]  (bottom: "A", top: "B")
```

Top of the stack is always the last item in the list.

Does this mapping feel clear?

abstract word “push” → Python code append

Reply with yes/no, and if anything feels fuzzy, tell me which part and I’ll slow it down with a small diagram next.

Yes, that feels clear

Nice 👌

Next piece: pop.

We already saw:

push → append → adds an item on top of the stack.

Now:

pop → pop → removes the top item and gives it back to you.

Example:

Python

```
stack = []
stack.append("A")   # stack: ["A"]
stack.append("B")   # stack: ["A", "B"]

top_item = stack.pop()   # removes "B"
# now: top_item == "B"
# and stack is ["A"]
```

So pop both:

Removes the last element
Returns it (so you can store it in a variable)
Before we do any coding:

Does it make sense that after stack.pop() on ["A", "B"], the list becomes ["A"] and the popped value is "B" (the old top)?

Yes, that makes sense

Perfect 👍

Let’s make this very concrete with a tiny step□by□step “plate stack” example.

Imagine we do these operations in order:

Start: stack = []

```
stack.append("plate1")
stack.append("plate2")
stack.pop()
```

I want you to mentally trace the list after each step:

After step 1: stack = ?
After step 2: stack = ?
After step 3: stack = ?
After step 4: stack = ? and the popped value is ?

Reply with your four answers (you can just write them as lines 1–4). Then we’ll move into the code editor and make the computer do it.

after 1 sytack is []
after 2 stack is ["plate1"]
after 3 stack is ["plate1", "plate2"]
after 4 stack is ["plate1"]

Nice trace, Ron ☑

You got all of these right:

[]
["plate1"]
["plate1", "plate2"]
["plate1"] and popped "plate2" (top plate)

That’s exactly how a stack should behave: last in, first out.

Next tiny idea: we’ll build a stack using a Python list and see it grow and shrink in real code.

I’ll guide you step by step in the editor.

Ready to switch to the code editor and write a small push/pop example together?

Yes, let's start coding

Awesome, let’s code this stack step by step

We’ll do three things:

Create an empty list to act as our stack
Push (append) a few items
Pop one item and print both the popped item and the final stack
Here’s the structure I want you to follow in the code editor (not in chat):

Start with:
stack = []
print("start:", stack)
Then push three items (your choice, e.g. "A", "B", "C"), using stack.append(...) and print after each push so you can see the growth.
Then:
top_item = stack.pop()
print("popped:", top_item)
print("after pop:", stack)
Go ahead and write this in the editor now, then hit Run to check it, and when it looks good, hit Submit code so I can review it.

Python

```
stack = []
print("start:", stack)
stack.append("X")
print("after push 1:", stack)
stack.append("Y")
print("after push 2:", stack)
stack.append("Z")
print("after push 3:", stack)
top_item = stack.pop()
print("popped:", top_item)
print("after pop:", stack)
```

Nice work, Ron

Your code is spot on:

Python

```
stack = []
print("start:", stack)
stack.append("X")          # push
stack.append("Y")          # push
stack.append("Z")          # push
top_item = stack.pop()     # pop
```

The output shows the stack growing and then shrinking, and top_item is "Z", the last

thing you pushed — perfect LIFO behavior.

Next tiny concept: sometimes we want to look at the top of the stack without removing it. That's called a peek, and in Python we can do it with:

Python

```
top = stack[-1]
```

This reads "last element in the list", which is the top of the stack.

Make sense that stack[-1] is just "peek at the top" (no removal)?

Yes, that makes sense

Great

Now I want to bring in an important safety rule: only peek or pop when the stack is not empty.

If the stack is empty ([]) and you try this:

Python

```
stack = []
top = stack[-1]      # or stack.pop()
```

Python will crash with an error, because there is no last item.

So we first check:

Python

```
if len(stack) == 0:
    print("stack is empty")
else:
    top = stack[-1]   # safe peek
    print("top is:", top)
```

len(stack) == 0 means: "the stack is empty".

Does this len(stack) == 0 check as an empty stack guard feel clear?

yes

What do you think that line is doing? Take your time with the question below

Suppose we have this Python code:

```
moves = []

# some code runs...

if len(moves) == 0:
    print("No moves to undo")
else:
    last_move = moves.pop()
    print("Undoing:", last_move)
```

What is this if len(moves) == 0: line doing?

A.

It checks if the stack moves is empty before popping.

B.

It always causes an error when the list is empty.

C.

It adds a new item to the stack if it is empty.

D.

It removes all items from the stack when it is empty.

Exactly right 🎉

That line:

Python

if len(moves) == 0:
is checking whether the moves stack is empty before calling moves.pop().

If it is empty → it prints "No moves to undo" and does NOT pop.
If it's not empty → it's safe to do moves.pop() and undo the last move.

This is the classic "underflow guard" for stacks: only pop when there's something there.

Want to try a slightly bigger example next, like a tiny "undo history" stack for typed text?

Yes, show me the undo example

Nice, let's build it step by step 😄⌨️

Imagine a tiny text editor. Every time the user types something, we push a snapshot of the text onto a stack called history. When they press Undo, we pop the last snapshot.

Here's the flow we'll implement:

Start with an empty history: history = []
Push a few states of the text, like:
"H"
"He"
"Hel"
"Hell"
"Hello"
Then "undo" once by popping from history, but only if it's not empty.
Print:
the state we're undoing to
the remaining history
Your task (in the code editor):

Write code that:

Creates history = []
Uses append to push at least 3 different text states
Prints the history after each push
Then:
If len(history) == 0: print "Nothing to undo"
Else: pop the top, store it in last_state, and print
"Undoing to:" and last_state
"History now:" and history

Don't write it in chat—put it in the editor, Run, then Submit code so I can review it.

Python

history = []

```
history.append("Rep")
print(history)
history.append("Repl")
print(history)
history.append("Reply")
print(history)
if len(history) == 0:
    print('Nothing to undo')
else:
      last_state = history.pop()
      print(f'Undoing to: {last_state}')
      print(f'History now: {history}')
```

This is excellent, Ron 🔥

Your code does exactly what a simple undo stack should:

Python

```
history = []
history.append("Rep")
history.append("Repl")
history.append("Reply")
```

You push each new text state onto history.
You print after each push, so you can see the stack growing.

Then you safely undo:

Python

```
if len(history) == 0:
    print('Nothing to undo')
else:
    last_state = history.pop()
    print(f'Undoing to: {last_state}')
    print(f'History now: {history}')
```

You correctly guard with len(history) == 0
You pop only when safe
Your output shows:
Undoing to: Reply
History now: ['Rep', 'Repl'] → top is now "Repl"

That hits all three goals:

build a stack from a list
grow/shrink it with append/pop
protect against empty stack ☑
Quick check before we think about wrapping this lesson:

If you now call history.pop() two more times in a row (starting from ['Rep', 'Repl']), what will the stack look like after each pop?

You can answer in words, like:

after 1st extra pop: ...
after 2nd extra pop: ...
affer 1st extra: ["Rep"]
after 2nd pop [] (but youll have to take my word for it ,illegal to peek lol)

🤣 That “illegal to peek” line is perfect.

And your answers are exactly right:

After 1st extra pop: ["Rep"]
After 2nd extra pop: [] (stack empty)

You've clearly got the growth, reduction, and empty check ideas down.

Before we wrap this lesson, one last tiny thing:

In your own words, which Python method do you use for:

push onto a stack
pop from a stack

You can just answer with two short lines like:

push → ...
pop → ...
i use list.append(item) to push
I use: storage_var = list.pop() to remove

What do you think? Pick the line that safely checks if moves is empty before popping 👇

We have a stack stored in a Python list named moves. Which line correctly checks if this stack is empty before we try to pop from it?

A.

if moves == []:

B.

if len(moves) == 0:

C.

if moves is None:

D.

if moves.pop() == 0:

Nice catch, Ron 👏

if len(moves) == 0: is exactly the right way to ask:

"Is this stack empty right now?"

len(moves) gives you how many items are in the stack.
== 0 means "no items" → empty → don't pop yet, avoid errors.

The other options either don't always work (moves == [] is okay but less common in this context) or are wrong (moves is None, moves.pop() == 0).

From what you've shown, you've nailed:

building a stack with a list
using append/pop to grow and shrink
checking emptiness before pop or peek

You've basically mastered this lesson ☑

Are you ready to wrap up this stack-using-lists lesson and move on to the next one, or do you want one more practice scenario first?

Wrap up and move on

Love it. Let’s close this one out cleanly ☑

Here’s what you can now do confidently:

Use a Python list as a stack
push → stack.append(item)
pop → top_item = stack.pop()
Mentally trace stack growth/shrink:
[] → ['X'] → ['X', 'Y'] → ['X'] → []
Safely check emptiness before touching the top:
if len(stack) == 0: → “nothing to pop or peek”
Build a simple undo history:
push states into history
pop to undo the last state
That’s solid stack intuition, and your “illegal to peek” joke tells me you really get the empty-stack idea 😄

You’re all set to move on.
When you’re ready, hit Next Lesson in the Learn tab, and I’ll be with you there to keep building on this.

```
history = []
history.append("Rep")
print(history)
history.append("Repl")
print(history)
history.append("Reply")
print(history)
if len(history) == 0:
    print('Nothing to undo')
else:
      last_state = history.pop()
      print(f'Undoing to: {last_state}')
      print(f'History now: {history}')
```